République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Département Génie Électrique
École Nationale d'Ingénieurs de Tunis

Commande neuronale d'un système thermique à distance

Étudiants : Ouri Mohamed Amine et Chaanbi Hassen
Encadrant : M. Faouzi Bouani

Année universitaire : 2025/2026

Plan de la présentation

Problématique et objectifs

Fondements théoriques

Validation expérimentale

Communication sans fil

Bilan et conclusion

Le problème

Comment commander un système thermique **non linéaire**, **de modéle inconnu** et **accessible à distance** ?

- PID → réglage difficile sans modèle précis
- Systèmes réels → non linéaires, perturbés
- Réglage empirique → instabilité possible
- Solution → **approches basées données (réseau de neurones)**

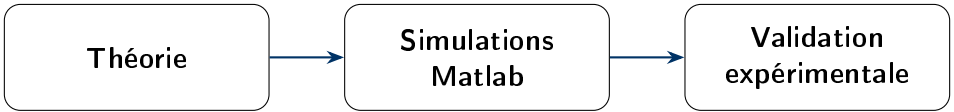
Pourquoi utiliser les RNA ?

Les réseaux de neurones permettent de commander des systèmes complexes **sans modèle mathématique explicite**.

- Approche basée sur les données expérimentales
- Capacité à approximer des systèmes **non linéaires**
- Adaptation aux variations et perturbations
- Réduction du besoin de modélisation analytique

Apprentissage d'une loi de commande à partir des données : entrée/sortie du système

Objectifs et démarche



- Etude des deux approches de commande
- Fondements théoriques
- Modélisation des systémes

- Simulations des deux commandes
- PID :Systéme du 1 et 2e ordre
- RNA :systéme non linéaire
- Analyse des performances

- Essai en boucle ouverte du systéme
- Extraction des données entrée-sortie
- Commande par esp32
- Commande à distance(ESP-NOW)

Commande PID : principe La commande PID (Proportionnelle - Intégrale - Dérivée) est une méthode de régulation basée sur l'erreur entre la consigne et la sortie.

- Erreur : $e(t) = r(t) - y(t)$
- Objectif : minimiser l'erreur et stabiliser le système
- Méthode simple et largement utilisée en industrie

Loi de commande

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}$$

$$u(k) = u(k-1) + c_0\, e(k) + c_1\, e(k-1) + c_2\, e(k-2)$$

Effet des actions PID

- **Action proportionnelle (K_p)**
  - Réagit instantanément à l'erreur
  - Améliore la rapidité
  - Peut provoquer un dépassement
- **Action intégrale (K_i)**
  - Élimine l'erreur statique
  - Améliore la précision
  - Peut ralentir le système
- **Action dérivée (K_d)**
  - Anticipe les variations de l'erreur
  - Améliore la stabilité
  - Sensible au bruit

Limite

Le réglage des gains (K_p, K_i, K_d) reste délicat pour les systèmes non linéaires.

Commande PID appliquée à un systéme de deuxiéme ordre

Fonction de transfert du système :

$$H(s) = \frac{K_s\,\omega_0^2}{s^2 + 2\xi\omega_0\,s + \omega_0^2}$$

K_s	5
ω_0	20 rad/s
ξ	0,5
T_e	0,36 s

$\xi < 1$: système sous-amorti $\Rightarrow$ présence de dépassement

Correcteur PID discrétisé :

$$u(k) = c_0\,e(k) + c_1\,e(k-1) + c_2\,e(k-2) + u(k-1)$$

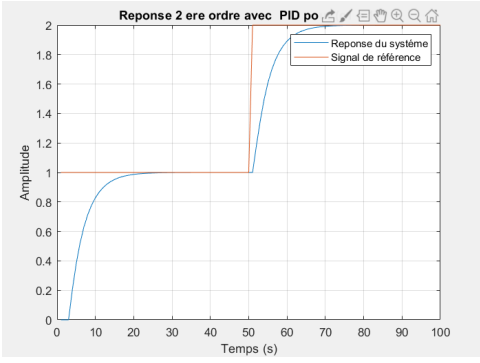
avec :

$$c_0 = K_p\left(1 + \frac{T_e}{T_i} + \frac{T_d}{T_e}\right)$$

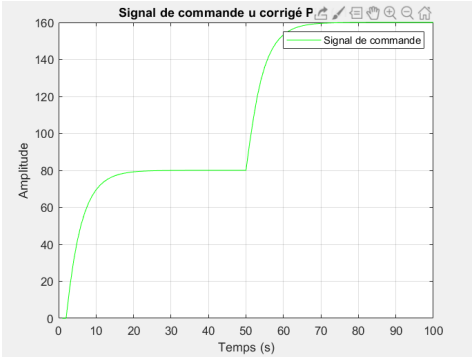
$$c_1 = -K_p\left(1 + \frac{2T_d}{T_e}\right), \quad c_2 = K_p\frac{T_d}{T_e}$$

c_0	15
c_1	1
c_2	1

Simulation PID – Résultats en boucle fermée



(a) Réponse du systéme et la consigne



(b) Signal de commande $u(k)$

$c_0 = 15,\ c_1 = 1,\ c_2 = 1 \quad \Rightarrow$**Suivi correct avec erreur statique nulle**

Principe

Un réseau de neurones (RNA) est un modèle mathématique capable d'approximer des relations non linéaires à partir de données.

- Approche basée sur les données (pas de modèle explicite)
- Approximation universelle des systèmes non linéaires
- Apprentissage à partir des couples entrée/sortie
- Utilisé comme modèle du système

Modélisation

$$\hat{y}(k) = NN\big(y(k-1), ..., y(k-n), u(k-1), ...u(k-m)\big)$$

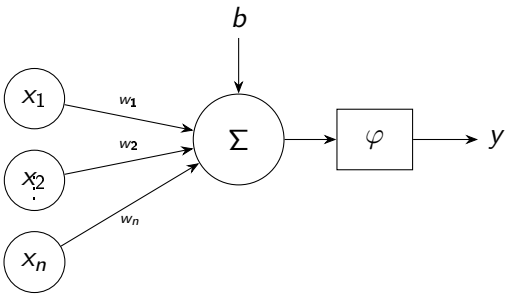
Intérêt des RNA

- capable d'apprendre des relations complexes essentiellement avec les systémes **non linéaires**
- Ne nécessite pas de modèle analytique précis
- Capacité d'adaptation aux perturbations
- Utilisé comme **prédicteur** dans la boucle de commande

Idée clé

Le RNA prédit la sortie future du système afin de calculer une commande optimale.

Neurone artificiel



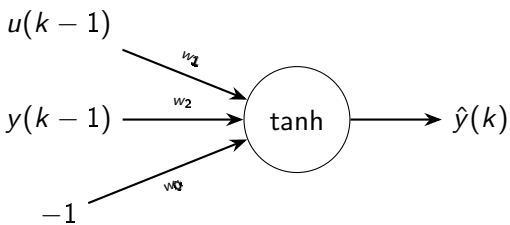
- Combinaison linéaire des entrées
- Ajout d'un biais
- Fonction d'activation non linéaire

Modèle mathématique

$$y = \varphi\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

- Exemple : $\varphi = \tanh$

Modélisation neuronale du système



- Modèle du système inconnu
- Entrées : $u(k-1)$, $y(k-1)$
- Sortie : $\hat{y}(k)$
- Approximation de la dynamique

Modèle

$$v(k) = -w_0 + w_1 u(k-1) + w_2 y(k-1)$$

$$\hat{y}(k) = \tanh(v(k))$$

Apprentissage du modèl neuronal :Identification du modéle neuronale du systéme

Objectif

Ajuster les poids du réseau pour minimiser l'erreur de prédiction

- Données : $(u(k), y(k))$
- Prédiction : $\hat{y}(k+1)$
- Erreur :

$$e(k+1) = y(k+1) - \hat{y}(k+1)$$

- Critère à minimiser :

$$J = \frac{1}{2}\sum_{k=k_0}^{N} e(k+1)^2 = \frac{1}{2}\sum_{k=k_0}^{N} \big[y(k+1) - \tanh\big(-w_0 + w_1 u(k-1) + w_2 y(k-1)$$

- Mise à jour des poids (descente de gradient) :

$$w_i \leftarrow w_i - \eta \frac{\partial J}{\partial w_i}$$

η : **pas d'apprentissage**

Algorithme d'apprentissage du réseau de neurones (BGD)

Entrées : $\{u(k), y(k)\}_{k=1}^{N}$ — **Sortie :** w_0, w_1, w_2

1. Initialiser les poids w_0, w_1, w_2
2. **Pour** chaque époque for $e = 1 \dots e_{\max}$: "Boucle de passage"

   2.1 chaque échantillon de la donnée for $k = 1 \dots N$: " Boucle de mesure "

$$\hat{y}(k) = \tanh(-w_0 + w_1 u(k-1) + w_2 y(k-1))$$

   2.2 Critère global :

$$C_r = \frac{1}{N} \sum_{k=1}^{N} (y(k) - \hat{y}(k))^2$$

   2.3 Gradient :

$$\frac{\partial C_r}{\partial w_j} = -\frac{2}{N} \sum_{k=1}^{N} (y(k) - \hat{y}(k))(1 - \hat{y}(k)^2)\varphi_j(k)$$

   2.4 Mise à jour :

$$w_j \leftarrow w_j - \mu_{\text{apprentissage}} \frac{\partial C_r}{\partial w_j}$$

Commande par Minimum de Variance

Objectif

Déterminer la commande $u(k)$ qui minimise l'erreur de poursuite.

- Utilisation du RNA comme modèl prédictif
- Prédiction de la sortie future $\hat{y}(k+1)$
- Minimisation d'un critère de performance

Démarche de calcul de la commande par minimum de variance

Le critére :

$$J = [y_c(k+1) - \hat{y}(k+1)]^2 + \lambda[u(k) - u(k-1)]^2$$

La descente de gradient permet alors de :

- corriger l'**erreur de poursuite** par rapport à la consigne $e(k+1) = [y_c(k+1) - \hat{y}(k+1)]$
- **pénalise les variations brusques** de la commande.

Minimisation de la commande :

$$u(k)_{i+1} = u(k)_i - \mu \frac{\partial J}{\partial u}$$

avec $v = -w_0 + w_1\, u(k-1) + w_2\, y(k-1)$ et $\hat{y}(k+1) = \tanh(v)$

Algorithme de commande par Minimum de Variance

Entrées : $y(k)$, $y_c(k+1)$, $u(k-1)$ — **Sortie :** $u(k)$

1. Initialiser : $x \leftarrow u(k-1)$
2. Calculer la prédiction courante :

$$\hat{y}(k) = \tanh\big(-w_0 + w_1\, u(k-1) + w_2\, y(k-1)\big)$$

3. **Pour** $i = 1 \ldots i_{\max}$:

   3.1 Prédire la sortie future corrigée :

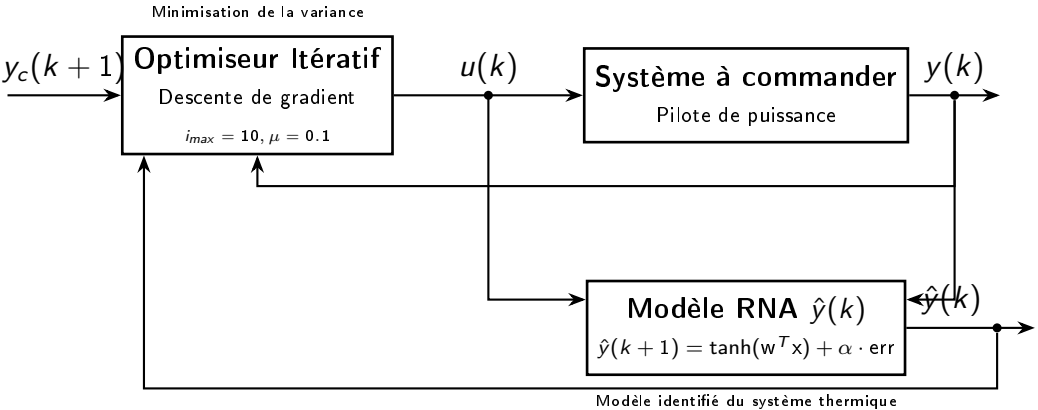
$$\hat{y}(k+1) = \tanh(-w_0 + w_1\, x + w_2\, y(k)) + \alpha\big(y(k) - \hat{y}(k)\big)$$

   3.2 Calculer le gradient :

$$\frac{\partial J}{\partial x} = -2\big(y_c(k+1) - \hat{y}(k+1)\big)(1 - \tanh^2 v)\, w_1 + 2\lambda(x - u(k-1))$$

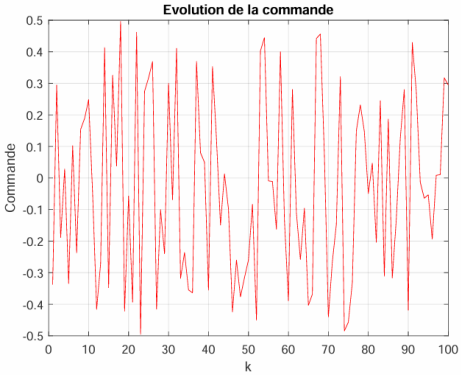
   3.3 Mettre à jour : $x \leftarrow x - \mu_{\text{commande}} \dfrac{\partial J}{\partial x}$

Architecture de la commande neuronale avec optimsateur par minimum de variance



Commande neuronale appliquée à un système non linéaire

$$y(k) = -0{,}8\, y(k-2)\, \frac{[u(k-1)]^2}{1+[u(k-1)]^2} + 0{,}033\, y(k-1) + 0{,}5\, u(k-1)$$



Topologie :

- Entrées : $[-1,\ u(k-1),\ y(k-1)]^\top$
- Sortie : $\hat{y}(k) = \tanh(\mathrm{w}^\top \mathrm{x})$

Non-linéarité + mémoire $(k-1, k-2)$
Commande aléatoire $u(k) \in [-0{,}5;\ 0{,}5]$

Résultats de l'identification neuronale :Phase d'apprentissage

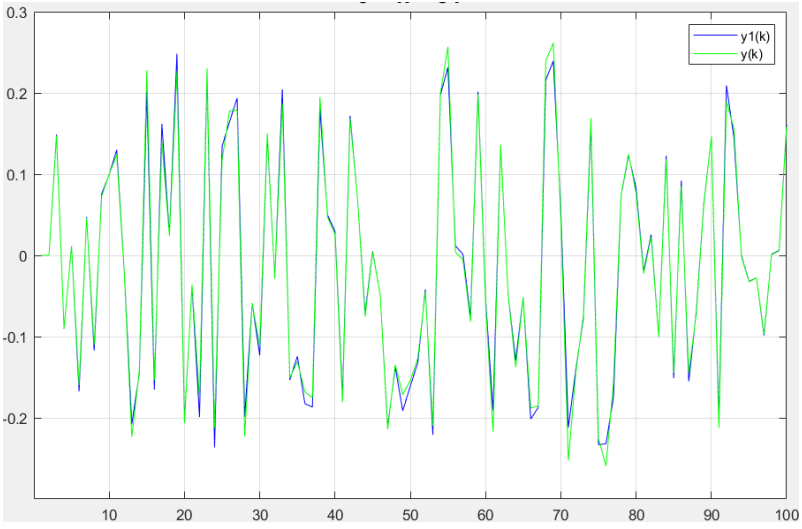
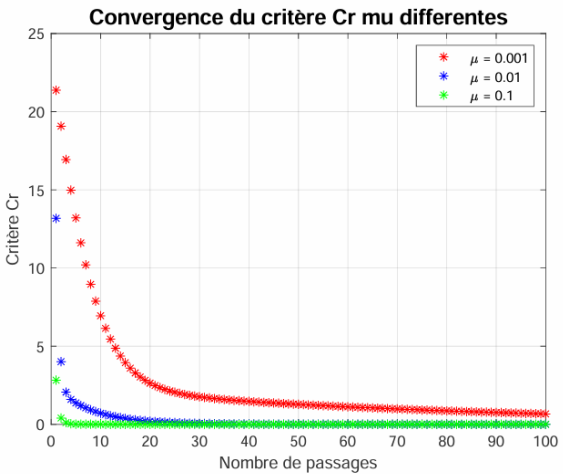


Figure –

Résultats de l'identification neuronale : Convergence du critère C_r (Descente de gradient BGD)



Convergence du critère C_r pour différents μ

Résultats de l'identification neuronale : Analyse de la convergence

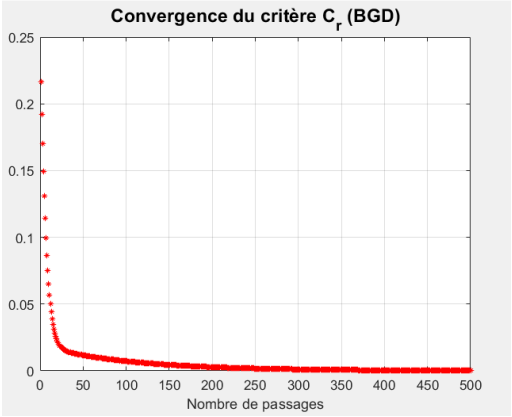
μ	Comportement	Verdict
0,001	Très lent	Trop lent
0,01	**Régulier**	**Optimal**
0,1	Rapide	Risque d'oscillations

Paramètres identifiés (pour $\mu = 0{,}01$)

$$w_0 = -0{,}0442 \qquad w_1 = 0{,}0121 \qquad w_2 = 0{,}9369$$

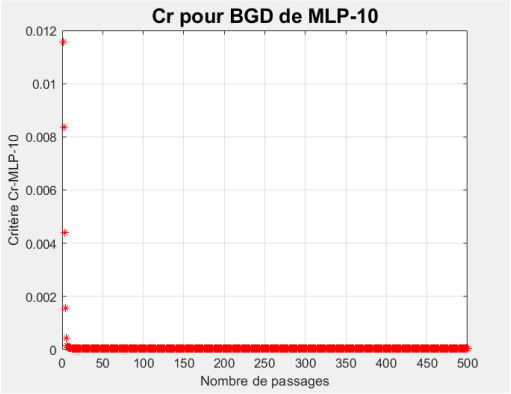
Comparaison : Convergence du critère C_r

1 Neurone



Convergence du critère C_r – 1 neurone

MLP – 10 neurones cachés



Convergence du critère C_r – MLP 10

Comparaison : Erreur finale – Phase de test

1 Neurone

```
Cr validation = 0.000037
>>
```

Phase de test – 1 neurone

Erreur de test

$$C_r^{\text{test}} \approx 2 \times 10^{-3}$$

- Écart visible entre y et $\hat{y}$
- Modèle **sous-paramétré**
- Ne capture pas la dynamique non linéaire

MLP – 10 neurones cachés

```
Erreur Test = 4.0812e-06
>>
```

Phase de test – MLP 10 neurones

Erreur de test

$$C_r^{\text{test}} = 4{,}08 \times 10^{-6}$$

- Courbes y et $\hat{y}$ **superposées**
- Bonne **généralisation**
- Dynamique non linéaire bien capturée

Régulation : Application de la commande par minimum de variance au système : Test avec une consigne variable par palier

Optimisation itérative ($i_{\text{max}} = 10$) :

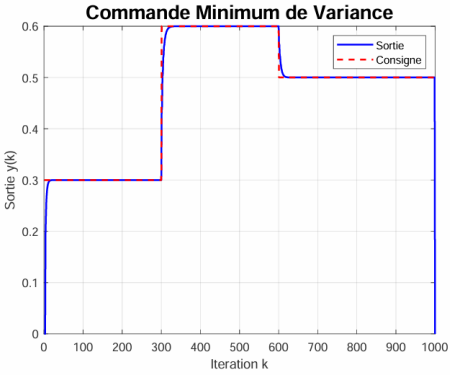
$$u_{\text{new}} = u_{\text{old}} - \mu \frac{\partial J}{\partial u}, \quad \mu = 0{,}2$$

Remarque :

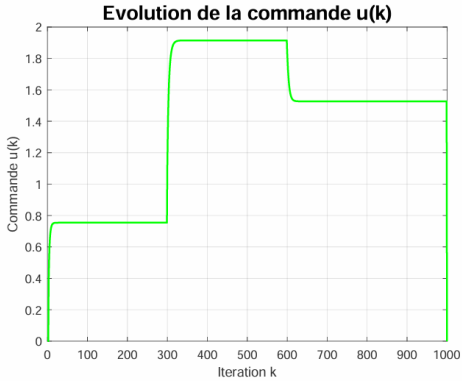
$$\mu_{\text{commande}} \neq \mu_{\text{apprentissage}}$$

λ petit $\Rightarrow$ convergence rapide $\quad | \quad$ λ grand $\Rightarrow$ convergence lente

Commande min. variance pour les valeurs optimisées de λ ($\lambda = 0.001$)



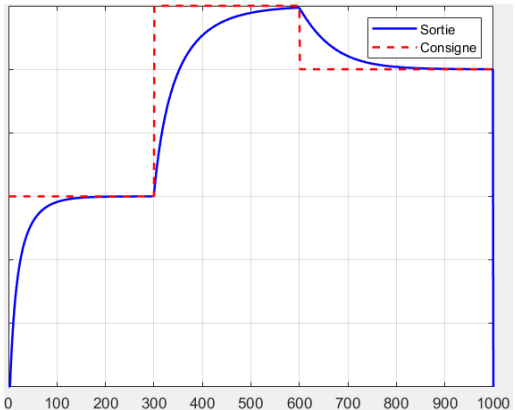
(a) Suivi de consigne



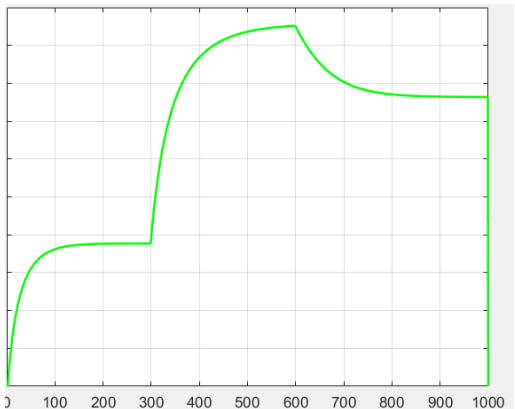
(b) Signal de commande $u(k)$

λ faible $\Rightarrow$ **suivi rapide**, variations de commande acceptées

Commande min. variance – $\lambda = 4$



(c) Suivi de consigne dégradé

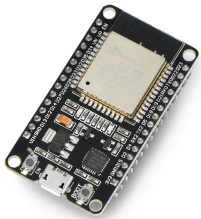


(d) Signal de commande $u(k)$

$\lambda = 4 \Rightarrow$ **commande lissée** mais suivi de consigne dégradé

III.Validation expérimentale

Matériels de l'essai



ESP32



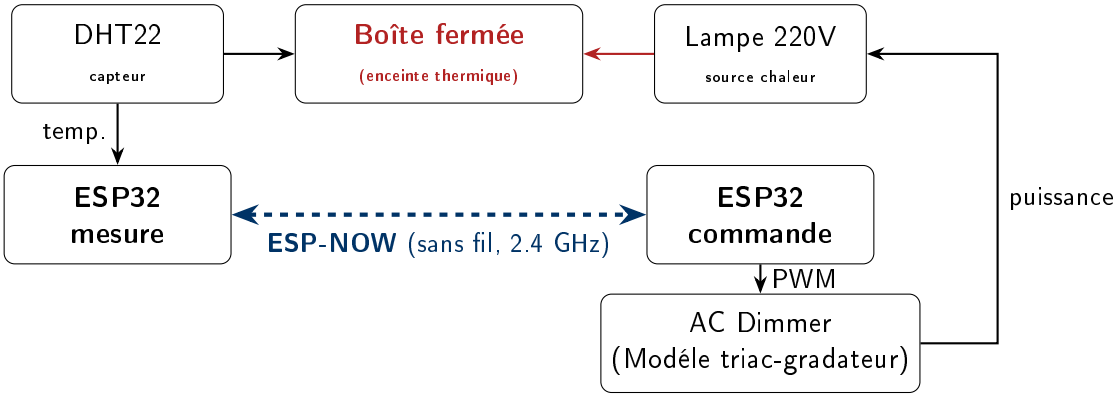
AC Dimmer



Capteur DHT22

	ESP32	**Dimmer**	**DHT22**
Rôle	Calcul & comm.	Puissance AC	Mesure temp.
Précision	–	8A, 220V	±0,1°C

Architecture du système



Essai en boucle ouverte :Collecte de données

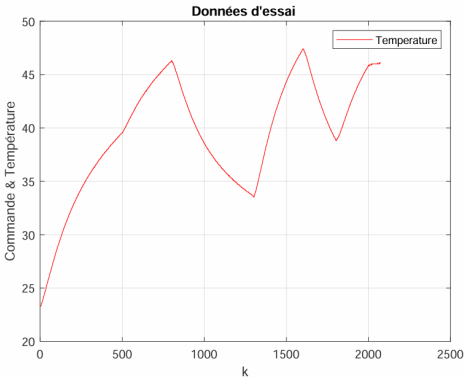
On lance un essai en boucle ouverte du système : le système est excité par une commande variable $u(k)$ avec une période d'échantillonnage $T_e = 2$ s, et la température est mesurée à chaque instant k.À l'issue de l'essai, on obtient une base de données expérimentale :

$$\mathcal{D} = \Big\{ \big(u(k),\ y(k)\big), \quad k = 1, 2, \ldots, 2000 \Big\}$$

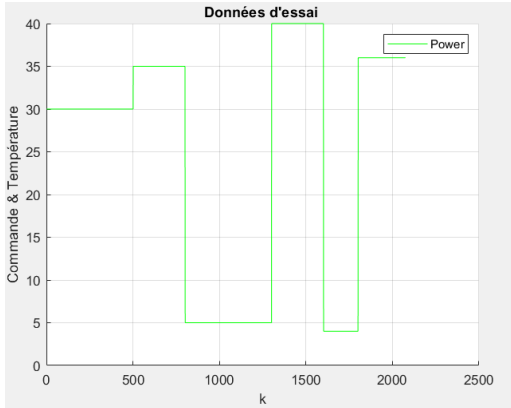
- $u(k)$: commande appliquée à l'instant k
- $y(k)$: température mesurée à l'instant k
- $T_e = 2$ s : période d'échantillonnage
- $N = 2000$: nombre total d'échantillons

$\Longrightarrow$ Cette base servira à **l'identification neuronale** du système.

Essai en boucle ouverte :Collecte de données

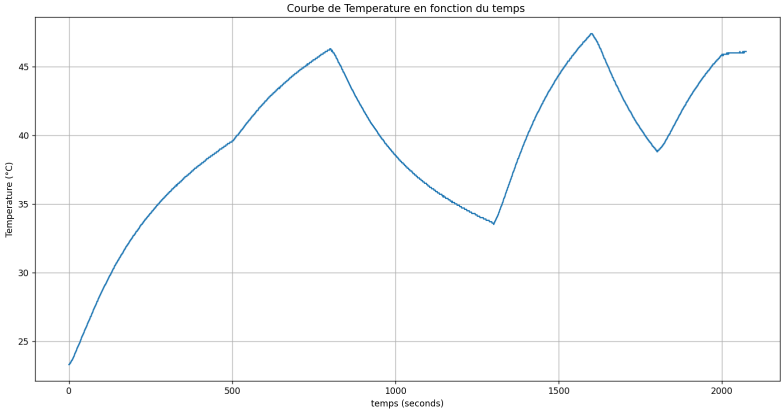


Température en fonction du temps



Commande appliquée

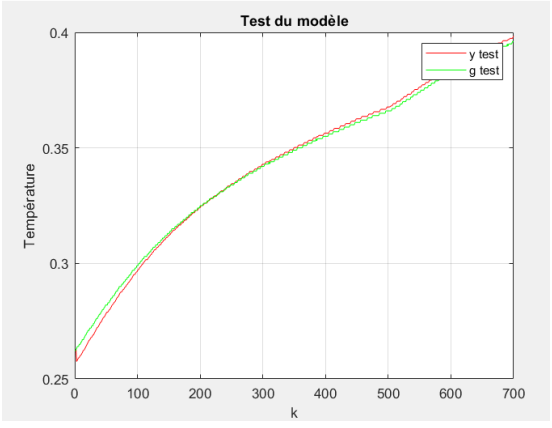
Apprentissage du neurone – Phase d'entraînement



Sortie système vs sortie prédite du neurone – **Phase d'apprentissage**

Données normalisées ($T/100$, $u/5$) | Descente de gradient itérative

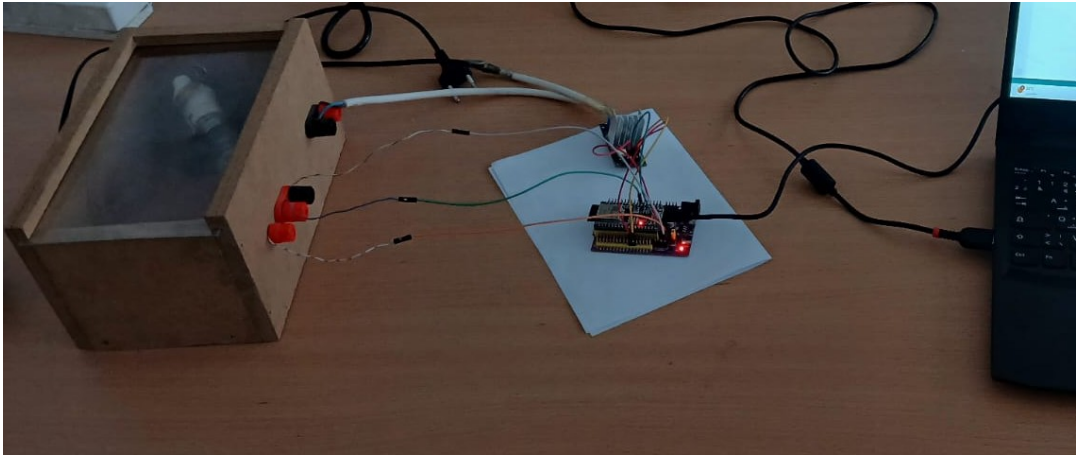
Apprentissage du neurone – Phase de test



Sortie système vs sortie prédite du neurone – **Phase de test**

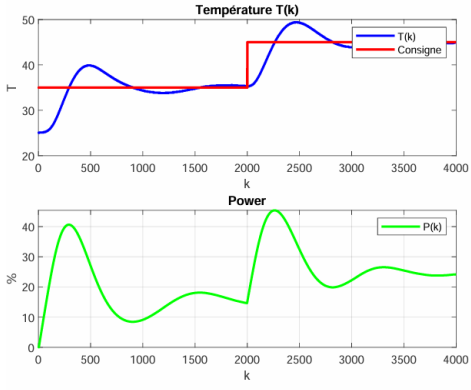
Le modèle **généralise correctement** sur des données non vues ⇒ poids validés

Montage expérimental – Boucle fermée



Montage : ESP32 + Dimmer + DHT22 + Lampe + Boîte fermée

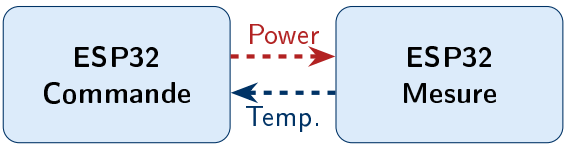
Résultats en boucle fermée – Commande câblée



Consigne 35°C puis 45°C – 4000 échantillons – Échantillonnage 2 s

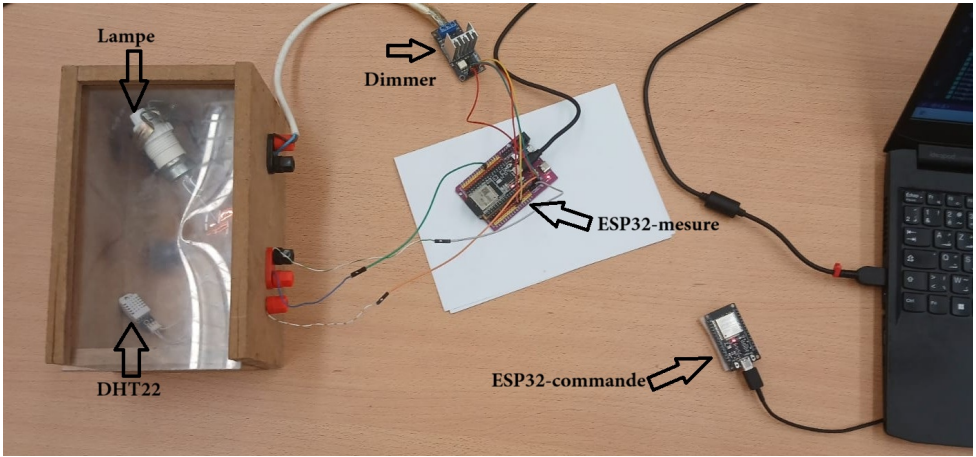
La température suit la consigne | Oscillations faibles | Système stable

Protocole ESP-NOW



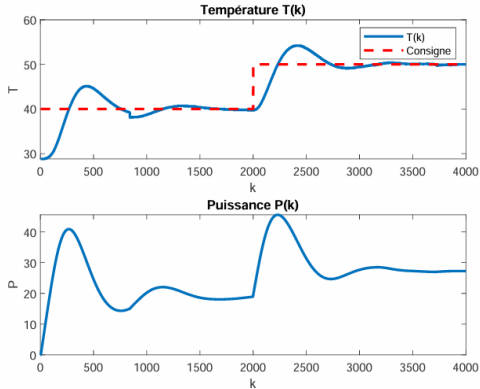
Fréquence	2,4 GHz
Latence	$\sim$ ms
Portée	$\sim$ 200 m
Wi-Fi requis	Non
Données max	250 oct/trame

Montage avec communication sans fil



Montage expérimental avec 2 cartes ESP32 communicant via ESP-NOW

Résultats – Commande à distance



Consignes : 40°C puis 50°C – Communication sans fil ESP-NOW

Suivi de consigne confirmé | Retard dû à l'inertie thermique | **Système stable**

Bilan comparatif

Critère	PID classique	Commande neuronale
Modèle requis	Oui (obligatoire)	**Non**
Syst. linéaires	Très efficace	Efficace
Syst. non linéaires	Inadapté	**Adapté**
Réglage	Manuel	Automatique
Adaptabilité	Faible	**Forte**

Conclusion clé

La commande neuronale est **complémentaire** au PID et **indispensable** pour les systèmes à modèle inconnu.

Conclusion

Ce qu'on a fait :

1. Fondements théoriques (PID vs RNA)
2. Simulations Matlab validées
3. Implémentation sur ESP32
4. Communication sans fil ESP-NOW
5. Validation expérimentale réussie

Perspectives :

- MLP multicouche plus profond
- Application à d'autres procédés industriels
- Commande prédictive neuronale
- Interface web de supervision
- Architectures avancées (LSTM)

Merci de votre attention !

Références et Bibliographie

- B. Tlili (2008). *Modélisation et commande des systèmes électriques*. Thèse de doctorat, ENSAP. https://theses.hal.science/tel00589735/file/THESE_B_TLILI_ACS_ENIT_2008.pdf
- Auteur (1998). *Méthodologie de modélisation et de commande par réseaux de neurones*. Thèse de doctorat, Université de Lille. https://pepite-depot.univ-lille.fr/LIBRE/Th_Num/1998/50376-1998-309.pdf
- K. Ben Saad (2020). *Cours d'Automatique générale*, ENIT. https://fr.scribd.com/document/493499963/cours-automatique-generale
- J. Haggége (2012). *Cours d'automatique*, ENIT. https://fr.scribd.com/document/311533640/cours-automatique-pdf

- M. Ould Moussa (2012). *Modélisation du comportement dynamique non linéaire des structures en MMF*. Thèse de doctorat, École Polytechnique.https://pastel.hal.science/pastel-00675919v1/file/modelisation_du_comportement_dynamique_des_MMFS.pdf
- B.Faouzi(2026). *Cours modélisation par réseaux de neurone artificiel*, École nationale d'ingénieurs de Tunis.
- K.Zohra(2022/2023). *Cours de l'automatique générale,introduction à la CNP*. Ecole nationale d'ingénieurs de Tunis. https://drive.google.com/drive/folders/1VLzkrtT62niSaIGkrXEUMNOMCWhLwC2g
- Claude Touzet(2016). *Les réseaux de neurones artificiels, Introduction au connexionnisme* https://amu.hal.science/hal-01338010/file/Les_reseaux_de_neurones_artificiels.pdf